

On-Device Training of Fully Quantized Deep Neural Networks on Cortex-M Microcontrollers

Mark Deutel¹, Frank Hannig¹, *Senior Member, IEEE*, Christopher Mutschler²,
and Jürgen Teich¹, *Fellow, IEEE*

Abstract—On-device training of deep neural networks (DNNs) allows models to adapt and fine tune to newly collected data or changing domains while deployed on microcontroller units (MCUs). However, DNN training is a resource-intensive task, making the implementation and execution of DNN training algorithms on MCUs challenging due to low processor speeds, constrained throughput, limited floating-point support, and memory constraints. In this work, we explore on-device training DNNs for different sized Cortex-M MCUs (Cortex-M0+, Cortex-M4, and Cortex-M7). We present a method that enables efficient training of DNNs completely in place on the MCU using fully quantized training (FQT) and dynamic partial gradient updates. We demonstrate the feasibility of our approach on multiple vision and time-series datasets and provide insights into the tradeoff between training accuracy, memory overhead, energy, and latency on real hardware. The results show that compared to related work, our approach requires 34.8% less memory and has a 49.0% lower latency per training sample, with dynamic partial gradient updates allowing a speedup of up to 8.7 compared to fully updating all weights.

Index Terms—Efficient AI, embedded machine learning, micro-controllers, on-device training, TinyML.

I. INTRODUCTION

ADAPTING, fine tuning, or retraining deep neural networks (DNNs) on microcontroller units (MCUs) to improve alongside a changing input domain or application is not a straightforward task [1]. Typically, once the DNN is deployed on the MCU, its trainable weights are stored read-only in Flash, and adapting the DNN without reprogramming the MCU is often not feasible. In addition, the limited memory and computational resources on the MCU require that the weights and feature maps of a DNN are often stored in quantized form, which further complicates regular DNN training normally performed in floating-point. Moreover, DNN

training itself adds significant memory and computational overhead, making on-device training on MCUs for DNNs of meaningful size and complexity even more challenging.

However, training DNNs on MCUs has a number of highly desirable properties, such as 1) a reduced communication of the MCU with remote training servers, since no data recorded by the MCU needs to be sent away for offline training; 2) increased privacy, since training with data from the input domain, which may contain personal information, can be performed directly on the MCU without any persistent storage on a remote server; 3) energy efficiency, since MCUs have an energy footprint much smaller than any GPU-based server; and 4) in place training with zero downtime, since DNN training can be performed on the fly alongside the regular inference of recorded samples on the MCU.

A. Challenges

Supervised training of DNNs relies on two algorithms: 1) backpropagation (BP), which is used to compute local gradients for each intermediate layer of the DNN based on the chain rule and 2) stochastic gradient descent (SGD), which is used to update the trainable weights of each layer based on the local gradients calculated by BP. Usually, the gradients used by SGD are stochastically approximated by accumulating them over a small subset of samples (minibatches) of all available training samples. The combination of BP and SGD is well-known, versatile, and the backbone of all major AI advances of recent years. However, it is also extremely memory- and compute-intensive, resulting in DNN training usually being performed on larger server systems and with computing accelerator cards like GPUs. This results in major challenges that need to be addressed to enable DNN training on regular MCUs.

The high computational cost of DNN training lies in the “backward pass,” i.e., the BP and SGD algorithms, that must be executed for each training sample in addition to the “forward pass,” i.e., the inference. The backward pass is responsible for propagating the error computed for the final layer of the DNN by a loss function to calculate local errors for all intermediate layers. As a result, for each layer, at least one partial derivative with respect to its input during the forward pass must be computed, while for all trainable layers another partial derivative with respect to their weights must also be computed to facilitate the parameter updates performed by SGD. For example, for linear layers and when using minibatches, this results in two additional transposed

Received 30 July 2024; accepted 12 October 2024. Date of publication 21 October 2024; date of current version 21 March 2025. This work was supported in part by the Bavarian Ministry for Economic Affairs, Infrastructure, Transport and Technology through the Center for Analytics-Data-Applications (ADA-Center) within the Framework of “BAYERN DIGITAL II,” and in part by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) (NA³Os) under Project 524986327. This article was recommended by Associate Editor M. Shafique. (Corresponding author: Mark Deutel.)

Mark Deutel, Frank Hannig, and Jürgen Teich are with the Department of Computer Science, Friedrich–Alexander–Universität Erlangen–Nürnberg, 91058 Erlangen, Germany (e-mail: mark.deutel@fau.de; frank.hannig@fau.de; juergen.teich@fau.de).

Christopher Mutschler is with the Fraunhofer IIS, Fraunhofer Institute for Integrated Circuits IIS, 91058 Erlangen, Germany (e-mail: christopher.mutschler@iis.fraunhofer.de).

Digital Object Identifier 10.1109/TCAD.2024.3484354

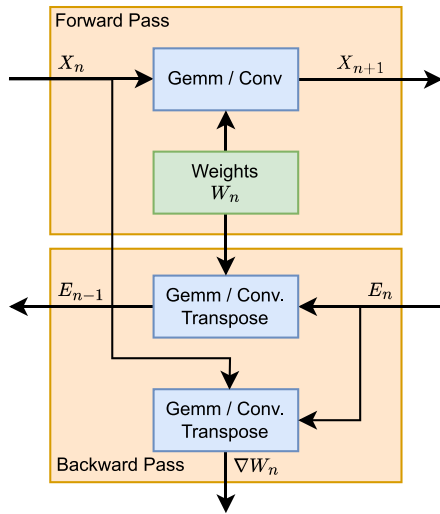


Fig. 1. Data dependencies between forward and backward pass of linear and convolutional layers in a DNN.

matrix multiplications, while for convolutional layers, two additional “transposed convolutions,” have to be performed in addition to the forward pass. In conclusion, for trainable layers in a DNN, the added computational cost of performing their backward pass is about twice as high as the computational cost of performing their forward pass.

The high memory overhead of DNN training is the result of four aspects as follows. First, The partial derivatives of most layers commonly used in DNNs depend on intermediate tensors used in the forward pass of the network, see Fig. 1 for a schematic overview. If only inference is performed, these tensors are usually short-lived because they only temporarily store data between successive layers. Therefore, to save RAM, many inference libraries for MCUs try to optimize memory requirements by reusing areas on the heap for multiple tensors during an inference pass [2], [3], [4]. However, when performing BP, many of these intermediate results need to be kept in memory longer, since they are required by operations in the backward pass that can only be executed after the complete forward pass has finished and the backward pass has propagated the error back to the corresponding layer. As a result, optimization of heap usage becomes less efficient.

Second, When performing SGD, many inference frameworks process an entire minibatch of samples at once, adding another “batch dimension” to the input, output, and all intermediate tensors, and thus greatly increasing the memory utilization, which now additionally depends on the minibatch size used.

Third, Additional memory must be provided to store training data over time, either as a labeled dataset for supervised training or a replay buffer for continual learning. On an MCU, this typically requires external memory from which data can be swapped in and out of RAM.

Fourth, Since the weights of all trained layers are dynamically updated at runtime, i.e., they are not read-only, they cannot be stored in Flash, where most MCU inference

libraries would place them, but must instead be placed in RAM.

B. Contribution

To address the challenges outlined in Section I-A and to make on-device training on tiny devices, such as Cortex-M MCUs feasible, we propose the following contributions in this article.

We introduce 1) an 8-bit fully quantized training (FQT) algorithm that quantizes both weights and intermediate feature tensors during the forward pass and errors and gradients tensors during the backward pass by using the same linear quantization scheme also used for regular DNN inference. As a result, our approach enables on-device DNN training on MCUs online, in place, and without any transformations, changes, or conversions to the DNN’s representation in memory or code between inference and training.

Furthermore, to allow FQT while still being able to achieve stable training results similar to regular floating-point training on GPUs, we propose 2) a memory-efficient version of minibatching with standardized gradients as well as 3) a method to optimize the distribution of values in the 8-bit range of quantized tensors by adapting the quantization parameters of trained weight tensors dynamically during training, similar to quantization-aware training (QAT) [5].

To further reduce the computational overhead of our approach, we employ 4) a dynamic gradient update strategy that is fully compatible with our FQT method, which prunes the computational tree of BP per training sample based on an online computed magnitude of error-based heuristic.

Finally, we demonstrate the flexibility of our approach for both transfer learning and full DNN training for multiple datasets (11 in total), and provide insights into the tradeoff between memory, latency, energy, and DNN accuracy for three Cortex-M-based MCUs, see Table II.

The remainder of this article is structured as follows. Section II outlines the work on-device training of DNNs on MCUs related to ours. In Section III we discuss our approach and its implementation in more detail. In Section IV we present our evaluation and then conclude in Section V.

II. RELATED WORK

On-device training on MCUs is gaining popularity as it promises a much simpler and more energy-efficient solution for DNN fine tuning and adaptation than training on a remote server and then redeploying the model. As a result, much research on-device training so far has focused on achieving a better tradeoff between memory overhead, computing resource requirements, and accuracy.

A popular approach to reduce memory utilization is to recompute intermediate results from the forward pass, which are required by the backward pass, instead of keeping them in memory [6], [7]. While this helps to minimize the additional memory overhead introduced by training, it does so at the cost of a significant additional computational overhead due to multiple computations of results. Another proposed approach

is to reduce the memory footprint of intermediate activation tensors by constructing a dynamic and sparse computational graph [8]. However, the approach as presented by the authors is only designed to minimize activations of the forward pass and still uses a matrix-vector product, albeit reduced in size, to decide which part of the computational graph to eliminate. Other approaches implement a hierarchical methodology with different levels of granularity at which to drop computations, such as the data, model, and algorithm levels [9]. Finally, reducing memory utilization during both inference and training by processing both activations and weights quantized has also been proposed [10], [11].

Due to the limited memory and computational resources on embedded devices, the most common DNN training tasks addressed by researchers so far are transfer learning and fine-tuning tasks, where an already well-trained DNN is adapted to a changed input domain or application. The reason for the popularity of these tasks is that they usually do not require training of a DNN from scratch but only fine tuning of its final layers. For example, Tiny-transfer-learning [12] freezes the weights of the DNN and trains only the biases, for which updates can be computed much easier and with less memory overhead than for weights. A similar approach is to train only batch normalization layers [13], [14]. Alternatively, TinyOL [15] trains only the final layer of the DNN to achieve reduced memory and computational overhead, while [14] introduces small additional “patches” of trainable weights to a well-trained DNN. However, while training only subsets of a DNN’s weights can lead to adequate training results, especially for more complex transfer learning tasks or larger domain shifts, the results can be poor [12], making these strategies not as reliable as full DNN training.

An approach to enable full DNN training on MCUs is described in [16], which uses two techniques: first, freeing up memory for activation tensors early at the cost of having to recompute them later, and second, paging where parts of the activation tensors are swapped out to a secondary, larger memory. However, swapping to external memory, in particular, is slow and makes certain assumptions about the MCU that may now be applicable in all cases.

Alternatively, Lin et al. [11] used a combination of quantized training, which they call quantization-aware scaling, combined with updating only parts of the weights on-device at runtime. This approach is probably the closest to ours, although there are some concerns that we try to address in this article. First, the approach described by the authors seems to be optimized exclusively for their own DNN architecture MCUNet [2], while our method focuses on general CNN support for different domains and applications. Second, the authors mostly provide simulated results for transfer-learning using their on-device training approach, while our work aims to explore the tradeoff between performance and resource requirements on MCUs of different scale (Cortex-M0+, Cortex-M4, and Cortex-M7) and for different tasks. Third, the gradient selection method proposed by the authors for their on-device training is performed entirely at compile time (offline). This effectively freezes parts of the DNN at runtime, which, similar to the concerns we mentioned about

transfer learning, can lead to poor training results for certain datasets and domains. To overcome this problem, our work proposes a method to dynamically evaluate which gradients to compute for each sample online during on-device training.

Finally, several memory-aware on-device learning frameworks have been proposed for larger embedded systems, i.e., mobile and IoT edge platforms [17], [18], [19], [20]. While inspiring, the methodologies proposed and implemented by these frameworks are not directly applicable to Cortex-M MCUs, mainly due to the much more constrained memory and compute resources present on MCUs compared to larger mobile and edge platforms.

III. CONCEPTS

To make supervised training feasible on MCUs, we propose a training framework centered around two techniques that we will discuss below: 1) FQT, see Section III-A and 2) dynamic sparse gradient updates, see Section III-B.

A. Fully Quantized Training

Typically, DNNs deployed on an MCU for inference are quantized using a linear quantization. This is done to save memory and because integer arithmetic can be performed more efficiently on most MCUs than floating-point arithmetic. Usually, quantization is implemented per tensor in the form of $v_q = \lfloor (v_f/s) \rfloor + z$, where s and z are the two parameters *scale* and *zero point* derived from the distribution of floating-point values in the original unquantized tensor. Therefore, in order to train a DNN in place without any conversion, training must be performed using the same quantization scheme already used for inference. In addition, this has the advantage that all optimizations already established for quantized inference can be reused for training.

A well-established method for training quantized DNNs on GPU-based systems is called QAT [5]. However, while the method helps to train DNNs robust to additional errors introduced by quantization, the DNN is not actually trained quantized, but still processed in floating-point arithmetic, with quantization only simulated, see Fig. 2(a) for a schematic overview. As a result, in QAT, the DNN is not fully transferred to quantized space until training is complete. Therefore, QAT does not provide any immediate memory or computational savings for the training process itself. In addition, using QAT would require a costly conversion process at the beginning and end of training.

To overcome the problems of QAT, and to achieve our goal of in place, online training on fully quantized DNNs, we propose a training method based on FQT, facilitating training directly with the quantized weights of a DNN and using the already established quantization scheme for inference, see Fig. 2(b). As a result, our method is flexible, since there is no difference in how the DNN is represented in memory and code between inference and training. Therefore, the same forward pass of a DNN executed on-device using our method can be used for both regular inference and training at the same time. Furthermore, since both the forward and backward pass of our training method are performed on quantized weights, feature

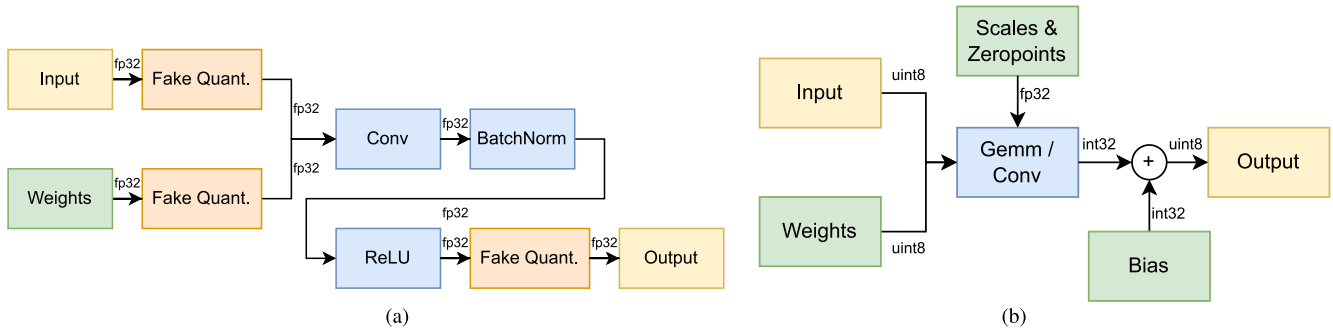


Fig. 2. Schematic comparison of a convolutional block with ReLU activation and Batchnorm for QAT and FQT. In FQT the convolutional, Batchnorm, and ReLU layers have been folded into a single monolithic QConv layer. (a) QAT. (b) FQT.

maps, errors, and gradients, the method is significantly more memory-efficient than traditional floating-point-based training.

BP for DNNs is based on the chain rule, and calculation of local errors and gradients for both linear and convolutional layers can be described as

$$E_{n-1} = W_n^T \cdot E_n \quad (1)$$

$$\nabla W_n = E_n \cdot X_n^T \quad (2)$$

with the $\cdot$ operator either being a matrix multiplication or a 2D-convolution. Therefore, (1) calculates the error E_{n-1} of the $n-1$ th layer of the DNN, given the error E_n and weight W_n of the n th layer, and (2) calculates the gradient ∇W_n used to update the weights of the n th layer given the error E_n and the input of the n th layer X_n , see Fig. 1. Note that, both (1) and (2) are based on the same mathematical operation $\cdot$ performed during the forward pass of the layer

$$Y_n = W_n \cdot X_n \quad (3)$$

with the addition of a transpose operator. In fact, (1) and (2) are the partial derivatives of (3) with respect to X_n and W_n , respectively. As a result, quantizing BP is straightforward, since the same quantization scheme and optimizations that are used in the forward pass during inference can be directly applied again. Therefore, for example, the fully quantized version of (1) for each element $e_{n-1} \in E_{n-1}$ can be computed from $e_n \in E_n$ and $w_n \in W_n$ as

$$e_{n-1} = \left[\frac{s_{w_n} s_{e_n}}{s_{e_{n-1}}} \sum (w_n - z_{w_n})(e_n - z_{e_n}) \right] + z_{e_{n-1}} \quad (4)$$

where s_{w_n} , s_{e_n} , and $s_{e_{n-1}}$ as well as z_{w_n} , z_{e_n} , and $z_{e_{n-1}}$ are the respective scale and zero point parameters for the quantized tensors E_n , E_{n-1} , and W_n . The quantized version of (2) looks identical, however in our approach we omit the requantization of ∇W_n since we perform gradient descent locally in floating-point space in the next step.

For updating each element of the quantized weight W_n of a given layer n from a current state i to a next state $i+1$ at the end of each mini-batch we use the gradient descent algorithm

$$w_{i+1} = \frac{1}{s_{w_{i+1}}} [(w_i - z_{w_i})s_{w_i} - \ell \nabla w_i] + z_{w_{i+1}} \quad (5)$$

adapted for quantization where ℓ is the learning rate, s_{w_i} and z_{w_i} are the current weight tensor's scale and zero point

parameters, and $s_{w_{i+1}}$ and $z_{w_{i+1}}$ are the updated weight tensor's parameters. To update the scale and zero point parameters alongside the trainable weights, $s_{w_{i+1}}$, see (6), and $z_{w_{i+1}}$, see (7), are calculated based on the maximum and minimum values $f_{\min}$ and $f_{\max}$ observed in the intermediate results from $(w_i - z_{w_i})s_{w_i} - \ell \nabla w_i$ which are in floating-point space

$$s_{w_{i+1}} = \frac{f_{\max} - f_{\min}}{255} \quad (6)$$

$$z_{w_{i+1}} = \left\lfloor -\frac{f_{\min}}{s_{w_{i+1}}} \right\rfloor. \quad (7)$$

To implement SGD, i.e., mini-batches, there are two viable approaches: 1) have all b input samples of a mini-batch in RAM at the same time and process them all at once during a single network pass by introducing an additional dimension to the input, output, and all intermediate tensors or 2) accumulate ∇W_n in a buffer over b successive BP steps of individual samples. Both options increase the amount of memory required compared to performing gradient descent on each sample, i.e., batch size $b = 1$. However, not using SGD and instead relying on per-sample updates can have a significant impact on the training performance of the DNN, as negative training effects, such as catastrophic forgetting [21] become much more prevalent.

We have chosen version 2) for our SGD approach. The reason is that by introducing additional gradient buffers to accumulate ∇W_n , the additional SRAM utilization to support SGD is independent of both the batch size b and the full network depth N , and instead depends only on the number of layers actually trained in the DNN. Most importantly, this approach does not require all the samples in a minibatch to be stored in RAM at the same time. Furthermore, to minimize the additional memory overhead introduced by SGD, we do not consider extensions, such as (Nesterov-) momentum or adaptive learning rate algorithms, such as Adam [22], as these techniques, while generally improving convergence and stability of training, come at the cost of significantly higher memory requirements due to the need to store multiple versions of the gradient buffer.

Finally, while testing with quantized BP and SGD, we found that standardizing the accumulated gradients per channel before performing the update step, similar to the intuition of RMSProp [23], improves the convergence stability of our FQT

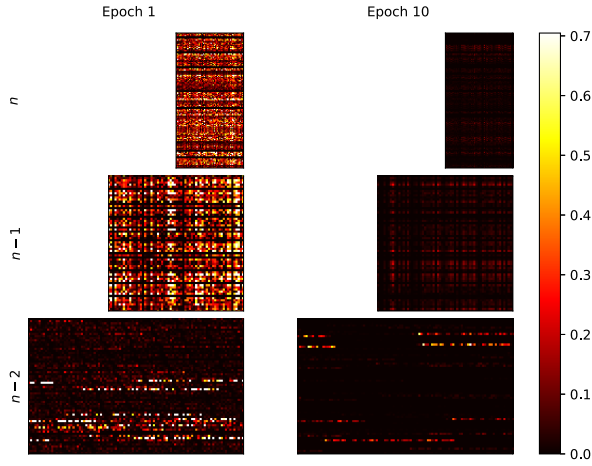


Fig. 3. Heatmaps of the absolute values of the gradient tensors of the final three linear layers of a DNN trained on the flowers dataset, exemplarily for a training sample after the first epoch (the left column) and the tenth epoch (the right column) of training.

approach. This results in a final per-element update step of our SGD approach in the form of

$$w_{i+1} = \frac{1}{s_{w_{i+1}}} \left[(w_i - z_{w_i}) s_{w_i} - \ell \frac{\nabla w_i - \mu_{\nabla W}}{\sigma_{\nabla W}} \right] + z_{w_{i+1}} \quad (8)$$

where $\mu_{\nabla W}$ and $\sigma_{\nabla W}$ are calculated as running mean and standard deviation of the local gradients ∇W calculated per sample.

B. Dynamic Sparse Gradient Updates

While quantized BP can help overcome the high memory overhead of BP, it does not directly contribute to reducing the algorithm's computational complexity.¹ Therefore, we propose *Dynamic Sparse Gradient Updates* as a technique that can be used in conjunction with our FQT-based training method to reduce the computational overhead added by BP with minimal impact on the training performance of our method.

We base our approach to only partially update gradients during BP on the hypotheses that 1) during BP, the magnitude of values in the gradient tensors becomes smaller; 2) during BP, the distribution of high magnitude values becomes sparser for earlier layer; and 3) as training progresses and the DNN has gained a good understanding of the underlying optimization landscape, different samples will only activate smaller clusters of neurons in a DNN at once. As a result, as training progresses, more and more values in backpropagated error tensors will not be of high magnitude, i.e., they will not lead to a significant update of the corresponding weights.

We illustrate the intuitions discussed above in Fig. 3, where we visualize the gradient tensors of the final three linear layers of a DNN trained on the flowers dataset from our evaluation. The heatmaps show the absolute values of the gradient updates of a single training sample after the first and the tenth epoch of training. We want to emphasize three

aspects. First, The gradient tensors become sparser during the backward pass per layer (compare rows n and $n - 2$). Therefore, our approach implements a gradient update rate for each layer. Second, In general, features are not learned randomly across neurons, but in structures that are activated at once, i.e., rows and columns for linear layers, features and channels for convolutional layers. This observation allows us to estimate the importance of parts of the gradient tensor, i.e., the magnitude of values, averaged per structure rather than per element, thereby decreasing the overhead of our method. Third, The overall magnitude of the gradient values decreases as training progresses (compare the two columns), resulting in fewer gradient structures worth updating in later stages of training than in early stages. As a result, we implement a dynamic update rate into our algorithm, which can decrease over the course of training.

Our proposed partial gradient updating algorithm monitors the absolute magnitude of the values in the error tensors of each layer for each training sample during BP as a simple heuristic to rank them. To limit the computational overhead of calculating the heuristic, our algorithm does not evaluate each value of the error tensors separately, but only observes the $l1$ -norm per channel/column. This optimization is based on the assumption that trainable DNN layers represent features in structures and not randomly, so it is worthwhile to update the weights of an entire structure when a high error is observed.

To calculate a perlayer update rate which can subsequently be used to select the total number of top- k values that should be kept in an error tensor based on the magnitude of error ranking achieved before, our approach uses the following equation:

$$k = \max\{\lfloor (\lambda_{\min} + |\varepsilon|(\lambda_{\max} - \lambda_{\min}))N \rfloor, 1\} \quad (9)$$

with $0 < \lambda_{\min} < \lambda_{\max} \leq 1$ being hyperparameters that control upper and lower percentage thresholds for the total amount of k structures to be kept. Furthermore, (9) considers $\varepsilon = (e/e_{\max})$ which is the fraction between the current layer's average error e and the maximum average error $e_{\max}$ observed for the layer over the whole training so far. As a result, our approach allows for a dynamic update rate between $\lambda_{\min}$ and $\lambda_{\max}$ over the course of training, where the higher the training success becomes, i.e., the smaller e becomes in relation to $e_{\max}$, the more the update rate will converge toward $\lambda_{\min}$.

IV. EVALUATION

We evaluate our approach on two training tasks: 1) transfer learning, see Sections IV-A–IV-C and 2) full DNN training, see Section IV-D. We consider three DNN configurations for all experiments: 1) *uint8* where the complete DNN is fully quantized; 2) *mixed* where the DNN is quantized except its classification head that remains in floating-point; and 3) *float32* where the DNN remains completely in its original floating-point representation as a reference. Since we observed in our experiments that especially the quantization of tensors in the final layers of the DNN can significantly affect on-device training, we chose the mixed configurations as another tradeoff between accuracy and resource utilization

¹Note that from a practical standpoint, just using FQT often already reduces the latency per training sample, since many MCUs can compute more efficiently with integers than with floats.

TABLE I
DATASETS CONSIDERED FOR TRANSFER LEARNING

Dataset	Classes	Input Size	Type
cwru [24]	9	1×512	Time Series
daliac [25]	13	1×1024	Time Series
speech [26]	36	1×16000	Time Series
animals [27]	10	$3 \times 128 \times 128$	Vision
cifar10 [28]	10	$3 \times 32 \times 32$	Vision
cifar100 [28]	100	$3 \times 32 \times 32$	Vision
flowers [29]	102	$3 \times 128 \times 128$	Vision

besides FQT. For all three DNN configurations, we used the micromod² platform and considered three MCUs for testing: 1) IMXRT2062; 2) nrf52840; and 3) RP2040, see Table II for detailed information.

A. On-Device Transfer Learning

To validate the effectiveness of our on-device FQT approach, we performed transfer learning on seven different datasets, see Table I for a detailed overview. We chose datasets of different complexity, input size and domains to show the versatility of our approach in enabling stable FQT.

We use the same DNN architecture for all the seven datasets which we dubbed *MbedNet*. Its architecture is closely related to MobileNetV3 [30] scaled down to be more suitable to the hardware constraints on MCUs. To facilitate time series data, we map the time dimension of input samples to one of the spatial input dimensions of the DNN while leaving the other spatial dimensions empty. This way, *MbedNet* can support nonimage input with only little changes to its architecture.

We conducted our experiments for three DNN configurations (uint8, mixed, and float). First, we trained a baseline model on ImageNet for all vision datasets. We then performed transfer learning on each of the target datasets, including network pruning to adapt the feature extraction of the model. We used an iterative pruning schedule based on an $L1$ -norm heuristic and automated gradual pruning. After training, we performed post-training quantization for both FQT and mixed training before deploying the models on the MCU. Once deployed, we set the final five layers of each DNN to random values, thereby resetting its classification capabilities. We then retrained the DNNs using the original training datasets for 20 epochs with a learning rate of 0.001 and a batch size of 48.

We report the results for all three DNN configurations and all tested datasets in Fig. 4. We show the accuracy on the test split after 20 epochs of retraining on-device in Fig. 4(a) (blue, orange, and green bars), averaged over five independent training runs and additionally provide the initial accuracy achieved by the model after GPU training (red bar, baseline). In cases where on-device training was not possible due to memory constraints, we simulated retraining on a desktop computer.³ For on-device training, we did not implement any data augmentation on the MCU, which for some datasets, e.g., speech or flowers, resulted in more pronounced overfitting

effects on device compared to the original GPU-based training, which included data augmentation.

For “simpler” problems with fewer classes, i.e., cwru, daliac, animals, and cifar10, we observed that on-device training was able to retrain or even improve the accuracy established by the baseline consistently and with almost no variance, while for “complex” problems with more classes, i.e., flowers, cifar100, and speech, retraining slightly underperformed compared to the other DNN configurations and the baseline (compare blue bars with the other bars). Due to the reduced quantized range of the 8-bit integer gradient tensors, ambiguous or subtle features may not be correctly learned or discriminated, resulting in less precise decision boundaries between classes. For all the datasets, this effect was easily resolved by training the classification head in floating-point, which resulted in a similar accuracy compared to full floating-point training (compare green and orange bars).

In Fig. 4(c) and (d), we give an overview of the on-device memory utilization for each of the seven datasets for transfer learning. To execute DNNs on-device, our training framework requires three separate segments of memory. Two segments in RAM, see Fig. 4(c) and one in Flash, see Fig. 4(d). The segments in RAM are required to 1) store feature maps, i.e., intermediate results of the DNN layers during both the forward and backward pass and 2) to store trainable weights and gradient buffers, while the segment in Flash memory is required for nontrainable weights. Since memory is one of the biggest limiting factors on all our target MCUs, see Table II, we have highlighted relevant memory constraints with red dashed lines both in Fig. 4(c) and (d).

The differences that can be observed in memory utilization of the DNNs between the datasets are 1) due to the different sample sizes of the datasets, resulting in differently sized input and feature tensors, see Table I and 2) the pruning we applied to the DNNs for some of the datasets during their initial training, resulting in smaller weight tensors. Compared to performing DNN inference on the device, additional RAM is required for weights and gradient buffers. Furthermore, the memory required for feature tensors may also increase, since intermediate results of layers must be kept longer in memory to facilitate the backward pass, resulting in fewer opportunities to reuse memory for subsequent intermediate tensors. In addition to trainable weights that must be stored in RAM, any weight tensor that is not to be retrained on-device can be stored read-only. Flash usage is shown in Fig. 4(d). We found that especially floating-point training could exceed the Flash of some MCUs, e.g., the nrf52840 in our experiments. In general, however, this was much less of a concern than managing RAM utilization.

In addition to memory, Fig. 4(b) shows the latency per training sample, i.e., the time between the DNN receiving an input, producing a prediction, and completing BP, as a unit of time. This latency is measured across all the datasets and DNN configurations that could be deployed on the IMXRT2062 MCU, taking into account its hardware limitations. All results are averaged over 1000 consecutive training samples and broken down by time spent in the forward and backward pass (compare light and dark colors). We observe that most

²<https://www.sparkfun.com/micromod>

³We used the same code and framework as used on the MCU minus some instruction set specific optimizations, which we had to disable.

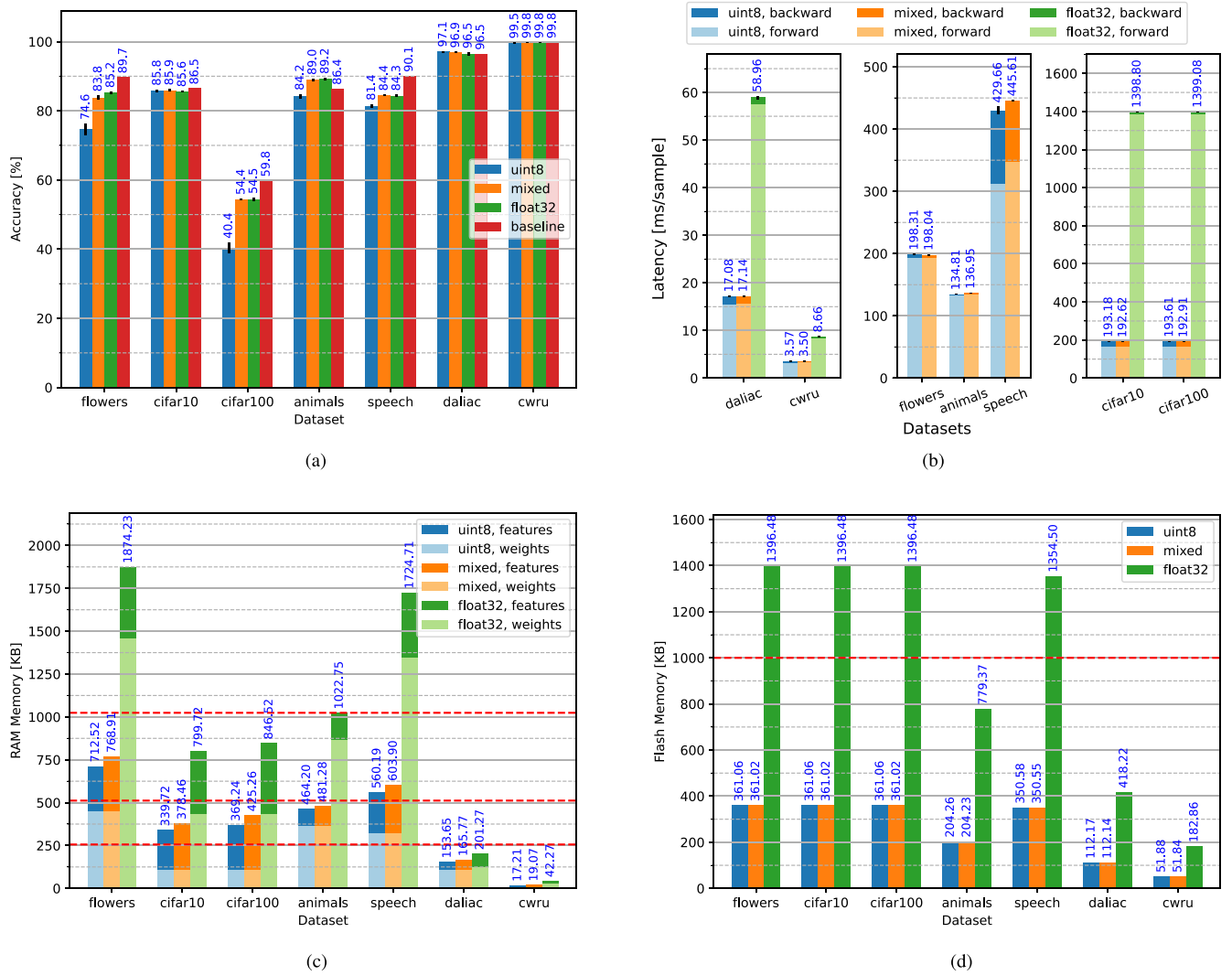


Fig. 4. Results of fully quantized on-device transfer learning (blue) compared to floating-point (green) and mixed training (orange). The accuracy results in Fig. 4(a) are averaged over five training runs. We also show baseline results trained on a GPU-based server in red. The latency results in Fig. 4(b) were measured on the IMXRT2062 MCU and are averaged over 1000 consecutive training steps. In both plots, we denote the standard deviation with black bars. Fig. 4(c) and (d) shows the memory utilization for Flash and RAM for all datasets as returned by our deployment framework. We have marked the memory constraints of the different MCUs with red dashed lines where relevant, see also Table II.

TABLE II
MCUS CONSIDERED FOR OUR EXPERIMENTS

Name	Type	Clock Speed	Amps Draw Idle	Flash/ROM	RAM
RP2040	Cortex-M0+	133 MHz	31.24 mA	16 MB (external)	264 KB
nrf52840	Cortex-M4	64 MHz	7.27 mA	1 MB (internal)	256 KB
IMXRT2062	Cortex-M7	600 MHz	108.26 mA	16 MB (external)	2×512 KB

of the training time was spent on the forward pass for transfer learning. This seems counterintuitive at first, since BP theoretically performs roughly twice as many computations as the forward pass. However, since we are performing transfer learning rather than full DNN training in our experiments, BP does not need to be executed for all layers of the DNN, only the final few. In addition, MBedNet is designed to learn compact representations quickly, resulting in large, computationally expensive initial layers, but compact, cheap final layers, allowing for a deep DNN architecture despite given resource constraints on MCUs. As a result, the time

spent computing forward through the expensive initial layers still dominates the computation of the cheap final layers, even with the added overhead of BP and training.

In conclusion, FQT significantly reduces memory requirements, making on-device training on MCUs feasible. Fine tuning the final layers comes at little additional latency cost, and even when quantized, full baseline accuracy can be retrained for most of the datasets we tested, where FQT performed subpar, using a mixed approach where the classification head is trained unquantized while convolutional layers are trained quantized consistently addressed this problem

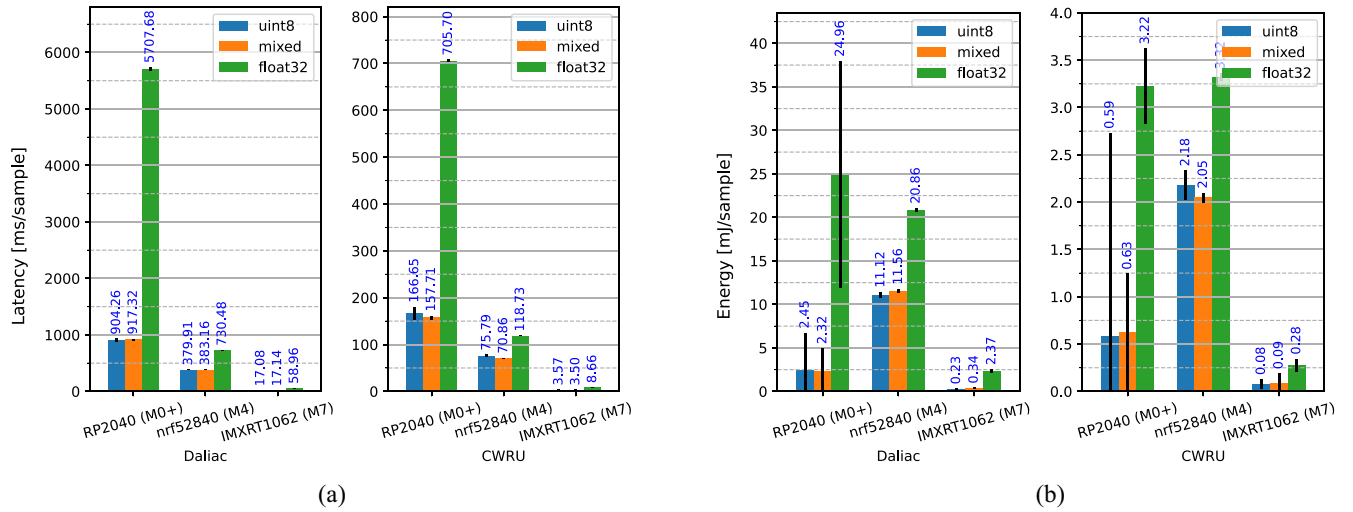


Fig. 5. Comparison of latency and energy for transfer learning on the CWRU and Daliac datasets for the three MCU platforms considered. All results are averaged over 1000 consecutive training steps, with the standard deviation shown as black bars. (a) Latency. (b) Energy.

while still offering an excellent tradeoff between memory utilization and latency.

B. Transfer Learning Across MCUs

As an additional experiment to the transfer learning task, and to get an understanding of the performance and constraints of FQT across different MCU platforms, we deployed the daliac and cwrU datasets on the nrf52840 and RP2040 MCU and compared our findings with the results we observed on the IMXRT2062, see Fig. 5. We only deployed the two datasets and not all seven because they were the only two that met the memory constraints on all three MCUs. As expected, processing on the IMXRT2062 is significantly faster than on the other Cortex-M4 and Cortex-M0+-based systems due to the much higher clock speed of the IMXRT2062, see Fig. 5(a). However, when comparing the RP2040 to the nrf52840 alone, even though the RP2040 has a higher clock speed, the nrf52840 was able to process training samples faster. This is because the nrf52840 has a dedicated floating-point unit (FPU) and implements the DSP instruction set extension, which our framework makes extensive use of to implement data-level parallelism (SIMD) in training sample processing, allowing much higher throughput than the RP2040, which has no FPU and no support for data-level parallelism.

In Fig. 5(b), we show the energy required per training sample averaged over 1000 consecutive training steps for each training configuration. Since we measured the entire MCU and not just the processor, we excluded the MCU's idle draw listed in Table II. On a per-sample basis, the IMXRT2062 is the most energy-efficient and the NRF52840 is the least. However, this analysis assumes that samples can be processed as quickly as possible without delay between samples. In many real-world applications, new samples arrive at a fixed rate, slower than the MCU can process them, which can cause it to sit idle the remainder of the time. As a result, in such scenarios, the MCU with the lowest idle power is usually the most energy-efficient,

which according to Table II would be the nrf52840 in our experiments.

C. Dynamic Sparse Gradient Updates

We validate the ability of our partial gradient updating approach to reduce the computational complexity of BP by comparing the training accuracy achieved with different dynamic gradient update rate configurations, see Fig. 6, with the results of our experiments presented in Section IV-A, where we used full gradient updates. We show the test accuracy for three different $\lambda_{\min}$ (blue: $\lambda_{\min} = 1.0$, orange: $\lambda_{\min} = 0.5$, and green: $\lambda_{\min} = 0.1$) for all DNN configurations and for all seven datasets considered in our transfer learning experiments, see Fig. 6(a)–(c). For all experiments, we performed on-device training in combination with dynamic gradient updates for 20 epochs, and we repeated each training five times and provide averaged results.

For both floating-point and mixed transfer learning [Fig. 6(a) and (b)], both $\lambda_{\min} = 0.5$ and $\lambda_{\min} = 0.1$ achieved the same level of accuracy after 20 epochs as $\lambda_{\min} = 1.0$, showing that, if chosen appropriately, only a small fraction of gradients need to be actually updated for each training sample to achieve a similar training result as with fully updated gradients. For FQT [Fig. 6(c)], $\lambda_{\min} = 0.5$ also worked without significant degradation compared to $\lambda_{\min} = 1.0$. However, with the extremely small update rate of $\lambda_{\min} = 0.1$, we observed that the training became unstable, i.e., high variance between different training runs, and performed significantly worse than $\lambda_{\min} = 1.0$.

To further demonstrate that introducing partial gradient updates actually reduces the computational complexity of BP and does not just result in a slower loss convergence over the course of training, we provide both loss and accuracy curves exemplarily for the flowers dataset and for the mixed DNN configuration for all three $\lambda_{\min}$ tested in Fig. 7. It can be seen that the loss convergence speed does not decrease for lower $\lambda_{\min}$ and is relatively consistent across all three different

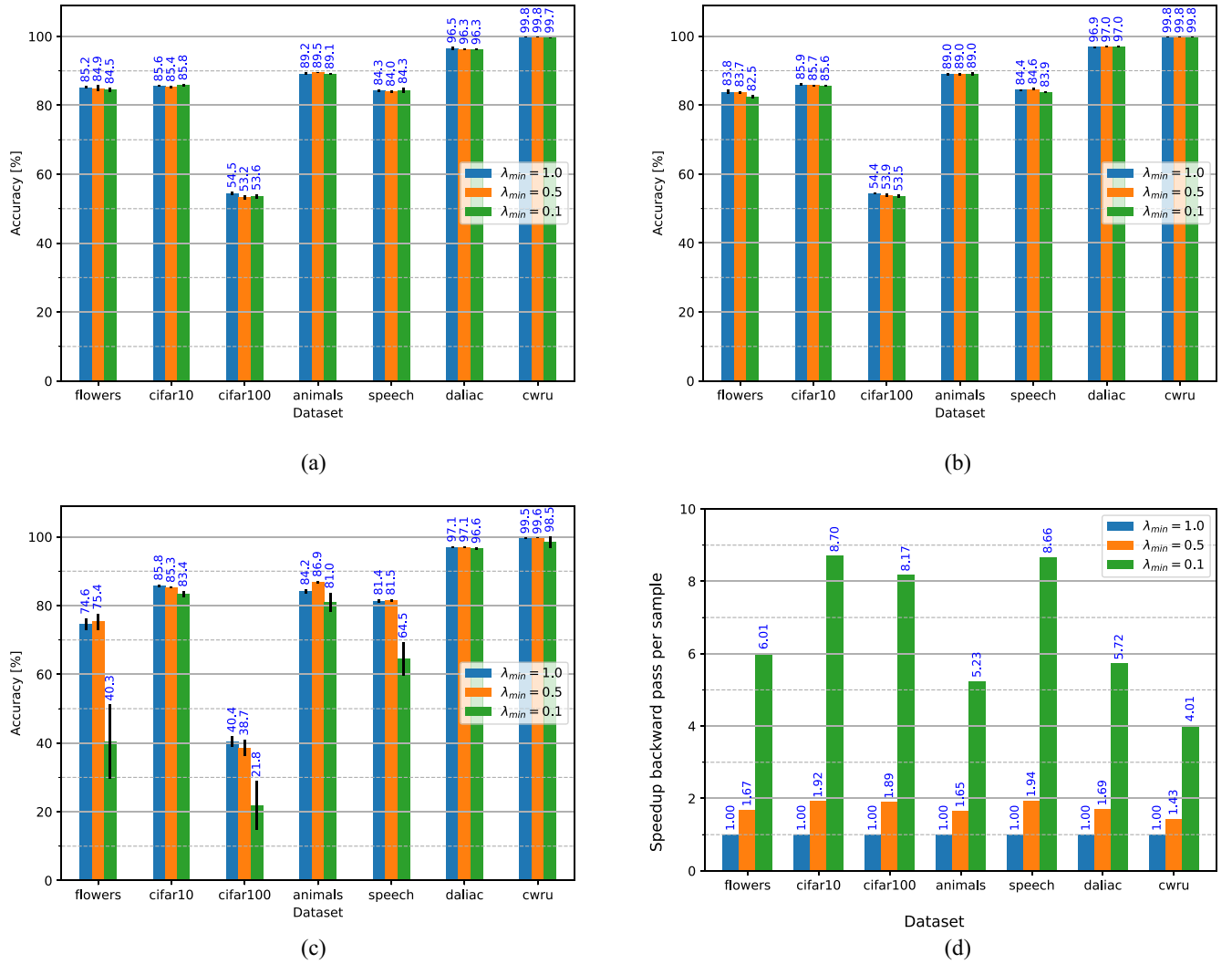


Fig. 6. Comparison of accuracy on the test datasets after 20 training epochs for three different gradient update rates ($\lambda_{\min} \in \{0.1, 0.5, 1.0\}$) for the mixed, quantized, and floating-point DNN configurations in Fig. 6(a) to (c). The results we report are averaged over five training runs. We show the standard deviation as black bars. Furthermore, we show the average speedup per sample for the different gradient update rates for the mixed training configuration in Fig. 6(d).

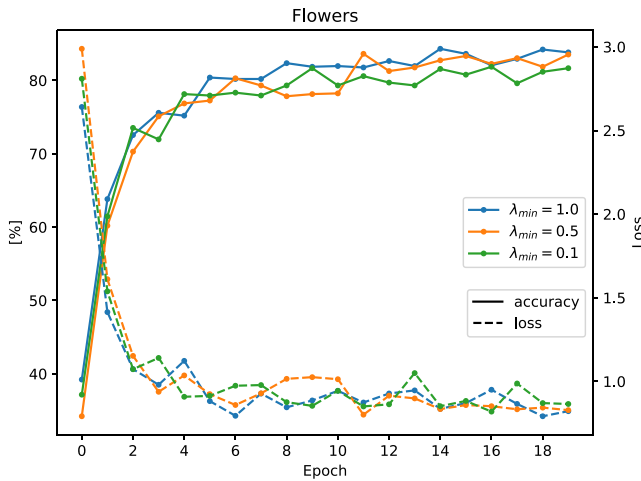


Fig. 7. Accuracy and loss curves for all three considered gradient update rates, exemplarily for a training run on the flowers dataset using the mixed DNN configuration.

gradient update rates, showing that dynamic gradient updating is able to reduce computational complexity while minimally impacting training performance.

Finally, we show the speedup achieved by partial gradient updating with all three $\lambda_{\min}$ considered when using the mixed training configuration on the IMXRT2062 for all seven datasets, see Fig. 6(d). We observed that for the mixed DNN configuration, reducing the number of updated gradients leads to a significant speedup per sample for BP, especially for $\lambda_{\min} = 0.1$, where an average speedup of about 6.64 was achieved across all the datasets. Since the overhead of partial gradient updating is the same for different $\lambda_{\min}$, while at the same time, a smaller $\lambda_{\min}$ leads to more operations being skipped, the increase in speedup is polynomial, depending on the number of loop nests in the layers trained.

D. Complete On-Device Training

To evaluate that our FQT approach can also be used to train DNNs on MCUs completely from scratch, we used a smaller DNN pretrained on the MNIST dataset [31], which we then fully retrained, i.e., all layers, on the device for four MNIST-related datasets, see Table III. The DNN we used for this purpose consisted of two convolutional layers, a max-pooling layer, and two linear layers, all with ReLU

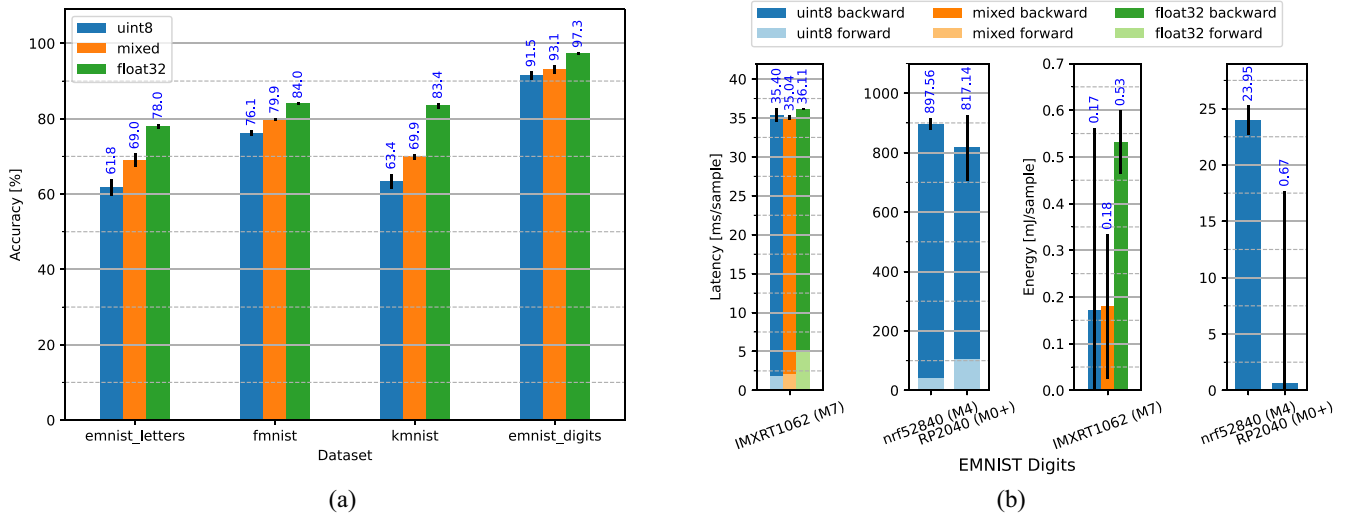


Fig. 8. Complete On-Device training of FMNIST, KMNIST, EMNIST Letters, and EMNIST Digits for a DNN consisting of two convolutional and two linear layers, both with ReLU and batch normalization, and with max-pooling in between. The accuracy results shown in Fig. 8(a) are averaged over five training runs, while the latency and energy results shown in Fig. 8(b) are exemplary for EMNIST for all three considered MCU platforms and are averaged over 1000 consecutive training steps. We denote the standard deviation with black bars.

TABLE III
DATASETS CONSIDERED FOR FULL ON-DEVICE TRAINING

Dataset	Classes	Input Size	Type
FMNIST [32]	10	$1 \times 28 \times 28$	Vision
KMNIST [33]	10	$1 \times 28 \times 28$	Vision
EMNIST-Letters [34]	26	$1 \times 28 \times 28$	Vision
EMNIST-Digits [34]	10	$1 \times 28 \times 28$	Vision

as activation and BatchNorm. We used the same DNN configurations as in Section IV-A, a learning rate of 0.001 and a batch size of 48. The corresponding results are shown in Fig. 8.

In Fig. 8(a), we show the accuracy results after on-device training for all three DNN configurations and for all four datasets. On-device full floating-point training achieved the highest accuracy, closely followed by mixed and FQT. Different from transfer learning, we observed a more pronounced gap between quantized and floating-point training. A reason for this observation is, that while the transfer learning experiments we discussed earlier only retrained the classification head and the final convolutional layers of the DNN, and thus could still rely on a well-trained feature extractor, i.e., the earlier layers, the full on-device experiments shown in Fig. 8(a) had to learn both feature extraction and classification from scratch. As a result, it appears that the limited resolution of the 8-bit data types used for training resulted in the DNN not being able to adequately learn the extraction of more complex features in its earlier layers. Therefore, although the training was still generally successful for FQT, we observed an earlier stagnation of FQT compared to floating-point or mixed training, and thus a slightly reduced accuracy. Similar effects can be observed with training of other kinds of low-bit representation DNNs like binarized DNNs [35]. A more fine-grained quantization scheme, such

as per-channel quantization [36] or alternative quantization strategies, such as weighted entropy [37] or stochastic quantization [38], while not as well supported by existing tool flows on MCUs, could be useful approaches to bridge this gap as long as they can maintain a reasonable computational overhead.

We also present latency and energy results for our complete on-device training experiments exemplarily for the EMNIST Digits dataset and for all three considered MCU platforms and training configurations in Fig. 8(b). For the nrf52840 and RP2040, we were only able to test the uint8 configuration due to memory constraints on both platforms. As in Fig. 4(b) above, we mark the fraction of latency per sample spent in the forward pass in light colors and in the backward pass in dark colors. Since we are training all layers of the DNN, the time spent in the backward pass is significantly higher than the time spent in the forward pass. These observations are the opposite of what we saw earlier in Fig. 4(b), where only the final layers were trained, but they are much closer to our initial intuition based on the theoretical analysis of FQT training that we stated at the beginning of this article. Nevertheless, the results presented in Fig. 8(b) clearly show the significant computational impact that especially training of early feature extracting layers has.

Finally, we show energy results in Fig. 8(b). Identical to the results discussed in Fig. 5(b), we subtracted the idle draw of the MCUs shown in Table II. As with the results discussed earlier, we can see that the IMXRT1062 is the most energy-efficient option per sample compared to the nrf52840 and RP2040, which instead have a much lower idle power draw.

E. Comparison With MCUNet-5FPS

We compare the performance of our FQT optimizer with the results reported by the authors of the optimizer

TABLE IV
COMPARISON BETWEEN OUR QUANTIZED OPTIMIZER AND SGD+M+QAS [11] FOR UPDATING THE FINAL TWO BLOCKS OF MCUNET-5FPS [2].
FOR OUR APPROACH, WE PROVIDE THE ACCURACY ON THE TEST SPLITS OF THE DATASETS

Precision	Optimizer	Accuracy (%) (MCUNet backbone: 23M MACs, 0.48M Param)								Avg. Acc.
		Cars [39]	CF10 [28]	CF100 [28]	CUB [40]	Flowers [29]	Food [41]	Pets [42]	VWW [43]	
fp32	SGD-M [11]	56.7	86.0	63.4	56.2	88.8	67.1	79.5	88.7	73.3
int8	SGD-M [11]	31.2	75.4	64.5	55.1	84.5	52.5	79.5	88.7	64.9
int8	SGD+M+QAS [11]	55.2	86.9	64.6	57.8	89.1	64.4	80.9	89.3	73.5
uint8	ours	54.5	89.5	65.2	58.5	85.8	66.6	79.8	89.3	73.7

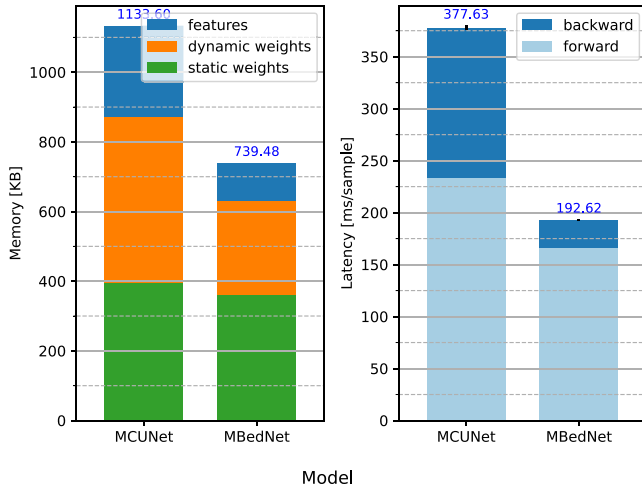


Fig. 9. Memory (green: ROM and orange + blue: RAM) and latency averaged over 1000 consecutive training steps of MBedNet (ours) compared to MCUNet-5FPS deployed on the IMXRT2062 for cifar10.

SGD+M+QAS [11] when training the final two blocks of MCUNet-5FPS [2], see Table IV. The first row in Table IV shows the baseline accuracy reported by [11] for MCUNet trained on eight datasets using a floating-point SGD optimizer with momentum (SGD-M). We could reproduce these results using the publicly available sources of MCUNet provided by the authors. The next two rows show the results obtained by [11] first for naively using quantized SGD-M without any changes besides the usage of quantized tensors and second for their SGD+M+QAS approach, which addresses shortcomings of the SGD-M approach. The final row shows the results we obtained for retraining MCUNet using our approach, i.e., FQT with standardized gradients and dynamic adaptation of the zero-point and scale parameters as described in Section III. All our reported results are on unseen test splits of the datasets. Like [11] we retrained for 50 epochs. We used a learning rate of 0.001 and a batch size of 48.

Our optimizer matched SGD+M+QAS closely, even outperforming it in five of the eight datasets, see the final two rows in Table IV. On average, this resulted in our approach having a slightly higher performance in retraining MCUNet, see the final column of Table IV, while exceeding the performance of naively retraining the quantized weights, i.e., SGD-M.

However, the main goal of both our optimizer and SGD+M+QAS is to provide a better tradeoff between memory and latency to meet the resource constraints of MCUs

and enable on-device training. But since Lin et al. [11] do not report detailed numbers for SGD+M+QAS, we instead report a comparison between MCUNet deployed with our framework and MbedNet for cifar10, see Fig. 9.

MCUNet has more trainable parameters than MBedNet, especially in its final layers. This results in 1) 34.8% less memory requirements and 2) 49.0% smaller latency of MBedNet compared to MCUNet, see Fig. 9. For memory, this results in 1) larger feature maps to propagate between layers (blue segment of bars) and 2) more memory required to store larger gradient and weight buffers to enable training of the final layers (orange segment of bars). This trend can likewise be observed for latency, see the left plot in Fig. 9. This is because MBedNet's final layers are smaller, both their forward and backward passes can be executed significantly faster than MCUNet's. Since the backward pass has to compute two partial derivatives for each layer, the difference is more pronounced than for the forward pass. Finally, when comparing accuracy, both architectures achieve a similar baseline of around 86%. When using our optimizer, MCUNet could be trained to an accuracy of 89.5% for cifar10, which is slightly higher than the 85.9% our optimizer achieved for MBedNet but which is in line with what SGD-M-QAS achieved on MCUNet (86.9%). Still, we argue that the significantly lower memory and latency cost of MBedNet compared to MCUNet makes it a more suitable architecture for efficient on-device training on MCUs. This allows MbedNet not only to run inference at around 5FPS (200 ms) like MCUNet but also to retrain.

V. CONCLUSION

In this work, we discussed the challenges of on-device training on Cortex-M microcontrollers, such as computational overhead and memory constraints, and presented a method to overcome them. To this end, we proposed a combination of fully quantized on-device training (FQT) combined with dynamic partial gradient updates to reduce compute overhead by dynamically adapting to the distribution and magnitude of gradient values during BP. In our evaluation, focusing on three different Cortex-M-based MCUs, we show that on-device transfer learning and full on-device training are both feasible from an accuracy and resource utilization standpoint. Therefore, our results indicate that on-device MCU training is possible for a wide range of datasets and applications. Furthermore, our method is flexible and allows for training configurations, where some layers of a DNN are trained in floating-point, while others are trained quantized. This

solidifies the capability of our method in adapting to problems of varying complexity and to different DNN architectures.

REFERENCES

- [1] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, and S. Han, "Tiny machine learning: Progress and futures [feature]," *IEEE Circuits Syst. Mag.*, vol. 23, no. 3, pp. 8–34, 3rd Quart., 2023.
- [2] J. Lin, W.-M. Chen, Y. Lin, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 11711–11722.
- [3] M. Deutel, P. Woller, C. Mutschler, and J. Teich, "Energy-efficient deployment of deep learning applications on cortex-M based microcontrollers using deep compression," in *Proc. MBMV 26th Workshop*, 2023, pp. 1–12.
- [4] C. Heidorn, F. Hannig, D. Riedelbauch, C. Strohmeyer, and J. Teich, "OpTC—A toolchain for deployment of neural networks on AURIX TC3xx microcontrollers," in *Proc. Stuttg. Int. Symp. Automot. Engine Technol.*, 2024, pp. 65–81.
- [5] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.
- [6] A. Gruslys, R. Munos, I. Danihelka, M. Lanctot, and A. Graves, "Memory-efficient backpropagation through time," in *Proc. 30th Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [7] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," 2016, *arXiv:1604.06174*.
- [8] L. Liu et al., "Dynamic sparse graph for efficient deep learning," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–18.
- [9] Y. Wang et al., "E2-train: Training state-of-the-art CNNs with over 80% energy savings," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–13.
- [10] N. Wang, J. Choi, D. Brand, C.-Y. Chen, and K. Gopalakrishnan, "Training deep neural networks with 8-bit floating point numbers," in *Proc. 32nd Adv. Neural Inf. Process. Syst.*, 2018, pp. 1–10.
- [11] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," in *Proc. 36th Adv. Neural Inf. Process. Syst.*, 2022, pp. 22941–22954.
- [12] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce memory, not parameters for efficient on-device learning," in *Proc. 34th Adv. Neural Inf. Process. Syst.*, 2020, pp. 11285–11297.
- [13] J. Frankle, D. J. Schwab, and A. S. Morcos, "Training BatchNorm and only BatchNorm: On the expressive power of random features in CNNs," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–28.
- [14] P. K. Mudrakarta, M. Sandler, A. Zhmoginov, and A. Howard, "K for the price of 1: Parameter-efficient multi-task and transfer learning," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–15.
- [15] H. Ren, D. Anicic, and T. A. Runkler, "TinyOL: TinyML with online-learning on microcontrollers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–8.
- [16] S. G. Patil, P. Jain, P. Dutta, I. Stoica, and J. Gonzalez, "POET: Training neural networks on tiny devices with integrated rematerialization and paging," in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 17573–17583.
- [17] Q. Wang et al., "Melon: Breaking the memory wall for resource-efficient on-device machine learning," in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Services*, 2022, pp. 450–463.
- [18] D. Xu et al., "Mandheling: Mixed-precision on-device DNN training with DSP offloading," in *Proc. 28th Annu. Int. Conf. Mobile Comput. Netw.*, 2022, pp. 214–227.
- [19] I. Gim and J. Ko, "Memory-efficient DNN training on mobile devices," in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Services*, 2022, pp. 464–476.
- [20] X. Qi and C. Liu, "Enabling deep learning on IoT edge: Approaches and evaluation," in *Proc. IEEE/ACM Symp. Edge Comput. (SEC)*, 2018, pp. 367–372.
- [21] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," *Psychol. Learn. Motiv.*, vol. 24, pp. 109–165, Jan. 1989.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017, *arXiv:1412.6980*.
- [23] G. Hinton, N. Srivastava, and K. Swersky, (Univ. Toronto, Toronto, ON, Canada). *Neural Networks for Machine Learning Lecture 6a Overview of Mini-Batch Gradient Descent*. Accessed: Jun. 4, 2024. [Online]. Available: <http://www.cs.toronto.edu/~hinton/coursera/lecture6/lec6.pdf>
- [24] (Case Western Reserve Univ., Cleveland, OH, USA). *Bearing Data Center*. Accessed: Mar. 8, 2024. [Online]. Available: <https://engineering.case.edu/bearingdatacenter>
- [25] H. Leutheuser, D. Schuldhaus, and B. M. Eskofier, "Hierarchical, multi-sensor based classification of daily life activities: Comparison with state-of-the-art algorithms using a benchmark dataset," *PLoS ONE*, vol. 8, no. 10, 2013, Art. no. e75196.
- [26] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, *arXiv:1804.03209*.
- [27] "Animals-10." Accessed: Mar. 8, 2024. [Online]. Available: <https://www.kaggle.com/datasets/alessiocrorad99/animals10>
- [28] A. Krizhevsky, *Learning Multiple Layers of Features From Tiny Images*, Univ. Toronto, Toronto, ON, Canada, 2009.
- [29] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *Proc. 6th Indian Conf. Comput. Vis., Graph. Image Process.*, 2008, pp. 722–729.
- [30] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1314–1324.
- [31] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [32] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [33] T. Clanuwat, M. Bober-Irizar, A. Kitamoto, A. Lamb, K. Yamamoto, and D. Ha, "Deep learning for classical Japanese literature," 2018, *arXiv:1812.01718*.
- [34] G. Cohen, S. Afshar, J. Tapson, and A. Van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2017, pp. 2921–2926.
- [35] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks," in *Proc. 30th Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [36] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," 2018, *arXiv:1806.08342*.
- [37] E. Park, J. Ahn, and S. Yoo, "Weighted-entropy-based quantization for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5456–5464.
- [38] Y. Dong, R. Ni, J. Li, Y. Chen, H. Su, and J. Zhu, "Stochastic quantization for learning accurate low-bit deep neural networks," *Int. J. Comput. Vis.*, vol. 127, pp. 1629–1642, Mar. 2019.
- [39] J. Krause, M. Stark, J. Deng, and L. Fei-Fei, "3D object representations for fine-grained categorization," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, 2013, pp. 554–561.
- [40] P. Welinder, S. Branson, T. Mita, C. Wah, and F. Schroff, "Caltech-UCSD birds 200," California Inst. Technol., Pasadena, CA, USA, Rep. CNS-TR-2010-001, 2010. [Online]. Available: https://www.vision.caltech.edu/datasets/cub_200_2011/
- [41] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101—mining discriminative components with random forests," in *Proc. 13th Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 446–461.
- [42] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. Jawahar, "Cats and dogs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3498–3505.
- [43] A. Chowdhery, P. Warden, J. Shlens, A. Howard, and R. Rhodes, "Visual wake words dataset," 2019, *arXiv:1906.05721*.